

1. Architecture Overview

The tool layer is split into three files to keep concerns separate:

- `src/tools/schemas.py` — Pydantic BaseModel definitions that describe every input field.
- `src/tools/note_tools.py` — Pure business logic that maps validated inputs to database calls.
- `src/tools/registry.py` — LangChain @tool wrappers that open a fresh per-thread SQLite connection and dispatch to `note_tools`.

Thread safety note: LangGraph executes ToolNode in a worker thread separate from the main graph. To avoid sharing a SQLite connection across threads, each tool invocation calls `_get_db_and_user()` to open and close its own connection.

1. Common Types

The following types are referenced throughout this document.

Type	Description
<code>str</code>	A Python string. Returned as a JSON string by all tools.
<code>list[str]</code>	A Python list of strings. Serialised as a JSON array.
<code>Optional[str]</code>	May be None / null. Omitting the parameter is equivalent to passing None.
<code>ISO 8601</code>	Date string in the form YYYY-MM-DD or YYYY-MM-DDTHH:MM:SS+00:00.
<code>UUID</code>	A version-4 UUID string, e.g. 3f7b2e1a-0a12-4c3d-8e9f-1234567890ab.
<code>JSON string</code>	All tools return a str that is a serialised JSON object or array. Parse with <code>json.loads()</code> if consuming programmatically.

1. Tool Reference

The agent exposes exactly five tools. They are listed below in the order they appear in `AGENT_TOOLS`.

4.1 create_note_tool

Purpose: Creates a new note and persists it to the SQLite database. After creation the note is optionally embedded for semantic search if the Ollama embeddings service is available.

Signature

```
create_note_tool(title: str, body: str, tags: list[str] = []) -> str
```

Parameters

Parameter	Type	Required	Description
<code>title</code>	<code>str</code>	Yes	A short, descriptive title for the note. Shown in list views and used in full-text search.

Parameter	Type	Required	Description
<code>body</code>	<code>str</code>	Yes	The main text content of the note. May be multi-line.
<code>tags</code>	<code>list[str]</code>	No	Optional list of category labels, e.g. ['meetings', 'urgent']. Defaults to an empty list.

Return Value

Returns a JSON string representing the newly created Note object. Example:

```
{ "id": "3f7b2e1a-...", "title": "Standup", "body": "Move to Tuesdays", "tags": ["meetings"], "created_at": "2025-04-22T10:00:00+00:00", "updated_at": "2025-04-22T10:00:00+00:00" }
```

Field	Description
<code>id</code>	UUID string. Uniquely identifies this note. Required for get, update, and delete operations.
<code>title</code>	The title as stored.
<code>body</code>	The body text as stored.
<code>tags</code>	JSON array of tag strings.
<code>created_at</code>	ISO 8601 timestamp of note creation (UTC).
<code>updated_at</code>	ISO 8601 timestamp of last modification (UTC). Equal to <code>created_at</code> on first creation.

Behaviour Notes

- A UUID is generated server-side; the caller cannot specify an ID.
- If the Ollama embedding service is reachable, the note is embedded immediately after creation so it is searchable via `semantic_query` in `list_notes_tool`.
- User isolation: the note is stored with the `user_id` injected via `RunnableConfig`. The LLM never sees or controls the `user_id`.

4.2 list_notes_tool

Purpose: Lists and searches the current user's notes. Supports four complementary filter modes: tag filter, full-text keyword search (SQLite FTS5), date range, and semantic similarity search. Filters can be combined except `semantic_query`, which takes precedence and short-circuits all other filters when provided.

Signature

```
list_notes_tool(tag=None, keyword=None, semantic_query=None, date_from=None, date_to=None) -> str
```

Parameters

Parameter	Type	Required	Description
<code>tag</code>	<code>Optional[str]</code>	No	Exact tag match. Returns notes whose tags list contains this string verbatim. Case-sensitive.
<code>keyword</code>	<code>Optional[str]</code>	No	Full-text search phrase. Matched against note titles, bodies, and tags using SQLite FTS5. Supports prefix matching via an implicit wildcard.

Parameter	Type	Required	Description
<code>semantic_query</code>	<code>Optional[str]</code>	No	Natural-language query for meaning-based search. Generates an Ollama embedding of the query and ranks stored notes by cosine similarity. Returns the top 5 results. Ignored if the embedding service is unavailable.
<code>date_from</code>	<code>Optional[str]</code>	No	Lower bound for <code>created_at</code> (inclusive). ISO 8601 format, e.g. 2025-01-01.
<code>date_to</code>	<code>Optional[str]</code>	No	Upper bound for <code>created_at</code> (inclusive). ISO 8601 format, e.g. 2025-12-31.

Return Value

Returns a JSON array of Note objects (same schema as `create_note_tool`) sorted by `updated_at` descending. When no notes match, returns:

```
{ "results": [], "message": "No notes found matching your criteria." }
```

When `semantic_query` is used and no embedding matches are found, returns:

```
{ "results": [], "message": "No semantically similar notes found." }
```

When `semantic_query` is used and results are found, each note object includes an additional field:

Field	Description
<code>_match_score</code>	Float between 0.0 and 1.0. Cosine similarity of the note's embedding to the query embedding. Only present on semantic search results.

Behaviour Notes

- Calling with no parameters returns all notes for the current user.
- `tag` and `keyword` can be combined with `date_from` / `date_to`. They cannot be combined with `semantic_query`.
- The agent must always call this tool before `update` or `delete` to verify the target `note_id` exists.

4.3 get_note_tool

Purpose: Retrieves a single note by its UUID. Use this when the agent already holds a valid `note_id` and needs the full note body, for example to display or compare content.

Signature

```
get_note_tool(note_id: str) -> str
```

Parameters

Parameter	Type	Required	Description
<code>note_id</code>	<code>str (UUID)</code>	Yes	The UUID of the note to retrieve. Must belong to the current user.

Return Value

On success: a JSON string of the Note object (same schema as `create_note_tool`).

On failure (note not found or wrong user):

```
{ "error": "Note <id> not found. You MUST call list_notes_tool to find the correct valid ID." }
```

Behaviour Notes

- The tool enforces user isolation: a note created by user A is invisible to user B even if the UUID is known.
- Prefer `list_notes_tool` when the `note_id` is unknown. Use `get_note_tool` only when the ID has already been confirmed.

4.4 update_note_tool

Purpose: Modifies one or more fields of an existing note. All fields are optional — only supplied fields are updated. After a successful update the note's embedding is regenerated if the embedding service is available.

Signature

```
update_note_tool(note_id: str, title=None, body=None, tags=None) -> str
```

Parameters

Parameter	Type	Required	Description
<code>note_id</code>	<code>str (UUID)</code>	Yes	The UUID of the note to modify. Must belong to the current user.
<code>title</code>	<code>Optional[str]</code>	No	New title. If omitted, the existing title is preserved.
<code>body</code>	<code>Optional[str]</code>	No	New body text. If omitted, the existing body is preserved.
<code>tags</code>	<code>Optional[list[str]]</code>	No	Complete replacement tag list. This fully replaces the existing tags array — it does not append. If omitted, tags are unchanged.

Return Value

On success: a JSON string of the updated Note object with a refreshed `updated_at` timestamp.

On failure (note not found):

```
{ "error": "Note <id> not found. You MUST call list_notes_tool to verify valid note IDs before replying." }
```

Behaviour Notes

- The agent must call `list_notes_tool` first to confirm the `note_id` is valid, per the system prompt rules.
- Passing `tags: []` clears all tags. Omitting tags entirely preserves them. These are distinct operations.
- Partial updates are safe: title, body, and tags may be mixed and matched freely in a single call.

4.5 delete_note_tool

Purpose: Permanently removes a note from the database. This operation is irreversible. The system prompt requires the agent to verify the `note_id` via `list_notes_tool` and obtain explicit user confirmation before calling this tool.

Signature

```
delete_note_tool(note_id: str) -> str
```

Parameters

Parameter	Type	Required	Description
<code>note_id</code>	<code>str (UUID)</code>	Yes	The UUID of the note to permanently delete. Must belong to the current user.

Return Value

On success:

```
{ "deleted": true, "title": "Standup Notes" }
```

Field	Description
<code>deleted</code>	Boolean true confirming the deletion succeeded.
<code>title</code>	The title of the deleted note, included so the agent can confirm the correct note was removed in its reply.

On failure (note not found):

```
{ "error": "Note <id> not found. You MUST call list_notes_tool to verify valid note IDs before replying." }
```

Behaviour Notes

- Deletion cascades: the FTS5 index is updated automatically via a SQLite trigger; no orphaned search entries remain.
- If an embedding exists for the note it will no longer be returned by semantic search after deletion.
- The agent must never call this tool speculatively. The required flow is: (1) call `list_notes_tool`, (2) confirm the note with the user, (3) call `delete_note_tool` only after explicit user confirmation.

1. Internal Helpers (Not Agent-Callable)

The following functions are internal to the tool layer. They are documented here for completeness but are not exposed to the LLM.

5.1 `_get_db_and_user(config)`

Defined in `src/tools/registry.py`. Opens a fresh SQLite connection for the current thread using the `db_path` from `config.configurable`, and extracts `user_id` from the same config. Returns a `(NoteDatabase, str)` tuple. Must be called at the start of every `@tool` function to ensure thread safety.

5.2 `embed_note(db, note_id, title, body, tags)`

Defined in `src/storage/embeddings.py`. Concatenates the note fields into a single text string, generates an embedding vector via Ollama (llama3.1), and persists it to the `notes.embedding` column. Called automatically after create and update if the embedding service is available.

5.3 `search_similar(db, query, user_id, top_k)`

Defined in `src/storage/embeddings.py`. Embeds the query string, loads all stored embeddings for the given `user_id`, computes cosine similarity for each, and returns the `top_k` (id, score) pairs sorted descending. Called by `list_notes_tool` when `semantic_query` is provided.

1. Error Handling

All tools return a JSON string in both success and failure cases. Error responses always contain an error key. The LLM is instructed by the system prompt to surface errors to the user in plain language and suggest corrective actions (typically calling `list_notes_tool` to find a valid ID).

Scenario	Response shape	Recommended agent action
Note not found	<pre>{ "error": "Note <id> not found..." }</pre>	Call <code>list_notes_tool</code> , tell user, do not retry with the same ID.
Semantic search unavailable	<pre>{ "results": [], "message": "... " }</pre>	Fall back to keyword or tag search and inform the user.
FTS5 syntax error	<pre>Empty list []</pre>	Inform user no results were found; suggest simpler keyword.
Database I/O error	<pre>Exception propagates to LangGraph</pre>	LangGraph logs the error. Agent informs user something went wrong.

1. Agent Behavioural Rules

The system prompt (`src/agent/system_prompt.py`) enforces the following rules governing when and how tools are called. These are reproduced here for reference.

#	Rule	Detail
1	Intent Disambiguation	If the user's request is ambiguous (multiple candidate notes), call <code>list_notes_tool</code> first, then ask the user to clarify before proceeding.
2	Confirm Destructive Actions	Before calling <code>delete_note_tool</code> , verify the note exists via <code>list_notes_tool</code> and obtain explicit user confirmation. Confirmation is also required for large rewrites via <code>update_note_tool</code> .
3	No Hallucination	Only reference note IDs, titles, or content retrieved from the database via a tool. Never invent note details.
4	Graceful Error Handling	If a tool returns an error or empty results, inform the user clearly and suggest an alternative action.
5	Multi-Turn Awareness	Resolve references like 'that note' or 'the last one' from conversation context before calling any tool.
6	Validate Before Acting	Even if the user supplies a <code>note_id</code> explicitly, call <code>list_notes_tool</code> to verify it exists before calling <code>update</code> or <code>delete</code> .
7	Strict Tool Call Format	Use the standard JSON tool-call structure. Never embed raw <code><function=...></code> XML in conversational output.

1. Note Data Model

All tools that return a note use the following field set, serialised by `Note.to_dict()` in `src/storage/models.py`.

Field	Type	Description
<code>id</code>	<code>string (UUID)</code>	Unique identifier. Use this value as <code>note_id</code> when calling <code>get</code> , <code>update</code> , or <code>delete</code> .
<code>title</code>	<code>string</code>	Short descriptive label.
<code>body</code>	<code>string</code>	Full text content of the note.
<code>tags</code>	<code>array of strings</code>	Category labels. May be an empty array.
<code>created_at</code>	<code>string (ISO 8601)</code>	Timestamp of initial creation, UTC.
<code>updated_at</code>	<code>string (ISO 8601)</code>	Timestamp of most recent modification, UTC. Automatically refreshed on every update.
<code>_match_score</code>	<code>float (0-1)</code>	Cosine similarity score. Present only in semantic search results from <code>list_notes_tool</code> .

1. RunnableConfig Keys

Tools receive contextual data through LangChain's `RunnableConfig` mechanism rather than as direct arguments. The following keys are expected under `config['configurable']`.

Key	Type	Description
<code>thread_id</code>	<code>string (UUID)</code>	Identifies the conversation thread for LangGraph's <code>MemorySaver</code> checkpointing. A new UUID is generated per session in <code>main.py</code> .
<code>db_path</code>	<code>string (path)</code>	File system path to the SQLite database, e.g. <code>data/notes.db</code> . Defaults to the <code>DB_PATH</code> environment variable.
<code>user_id</code>	<code>string</code>	Scopes all database reads and writes to a single user. Set server-side; the LLM never controls this value. Defaults to <code>'default'</code> .

1. Document History

Version	Date	Notes
1.0	April 2025	Initial release. Documents all five agent tools, internal helpers, error handling, behavioural rules, and data model.